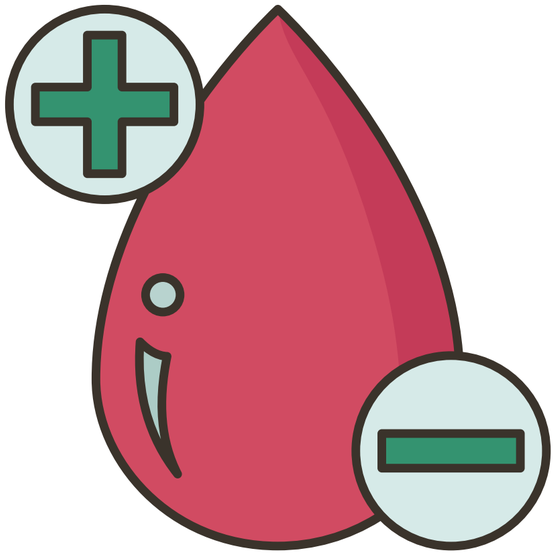
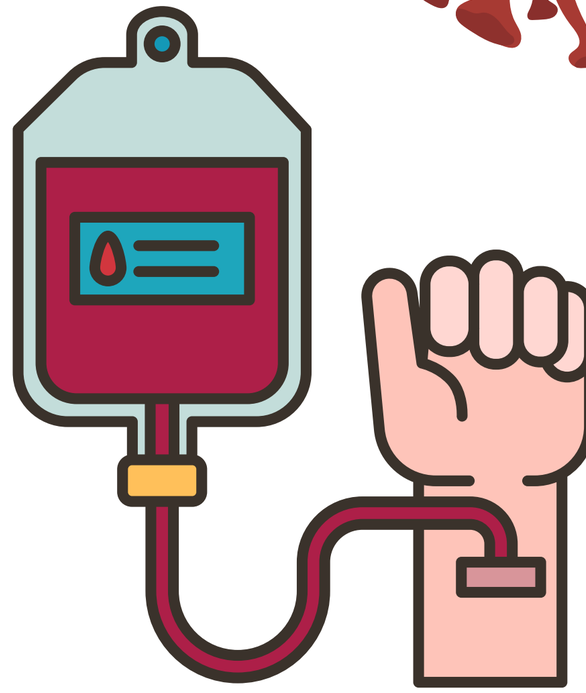
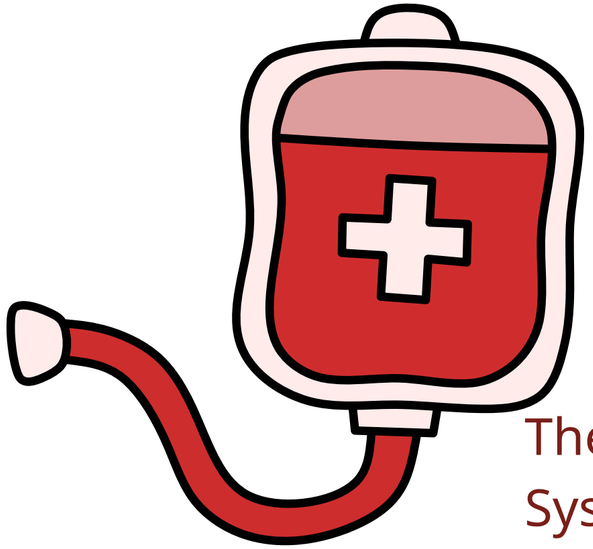
CAPSTONE PROJECT
BLOOD BANK INVENTORY & DONOR MATCHER



Presented by
2620030450_Aslesha L
2620030449_R Naga Tharini

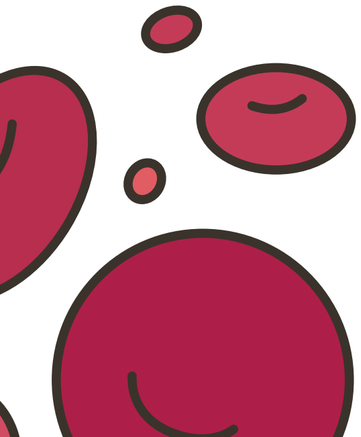


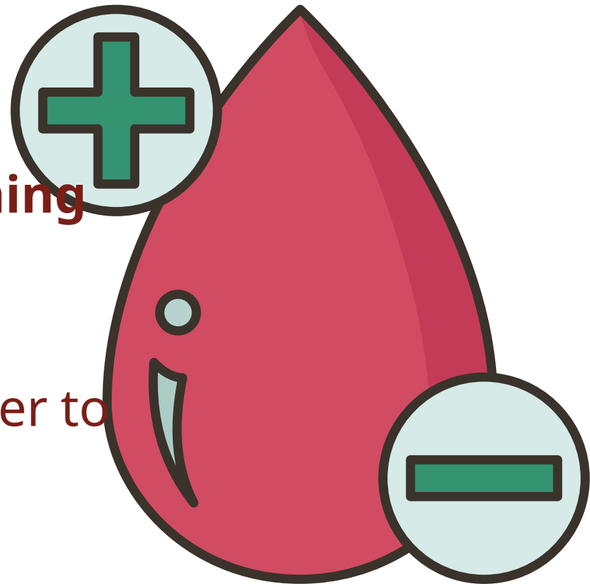


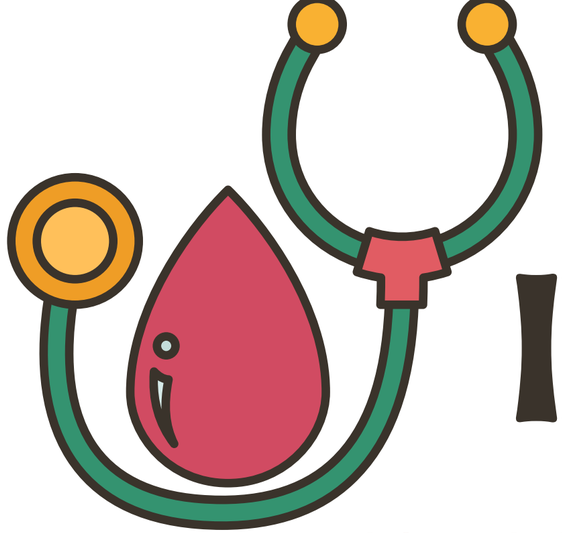
ABSTRACT

The Smart Blood-Stock Management and Donor Identification System is a Java-based application designed to efficiently manage blood-stock information and identify suitable blood donors when required.The system helps maintain details such as blood group, available blood units, donor information, and donor eligibility.

The project is developed using **Object-Oriented Programming** (OOP) concepts in Java, including classes and objects, and abstraction.These concepts help organize the system into separate and reusable modules, making the application easier to understand, maintain, and extend.

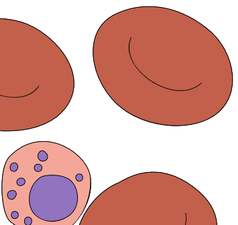


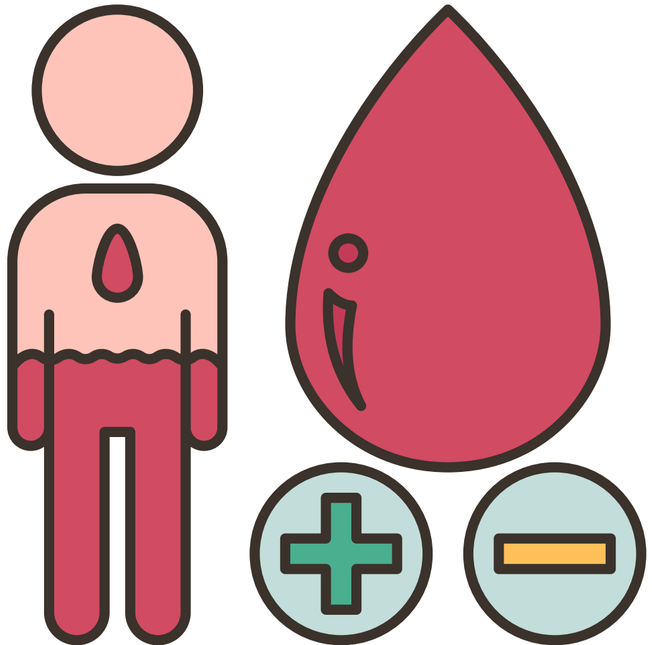




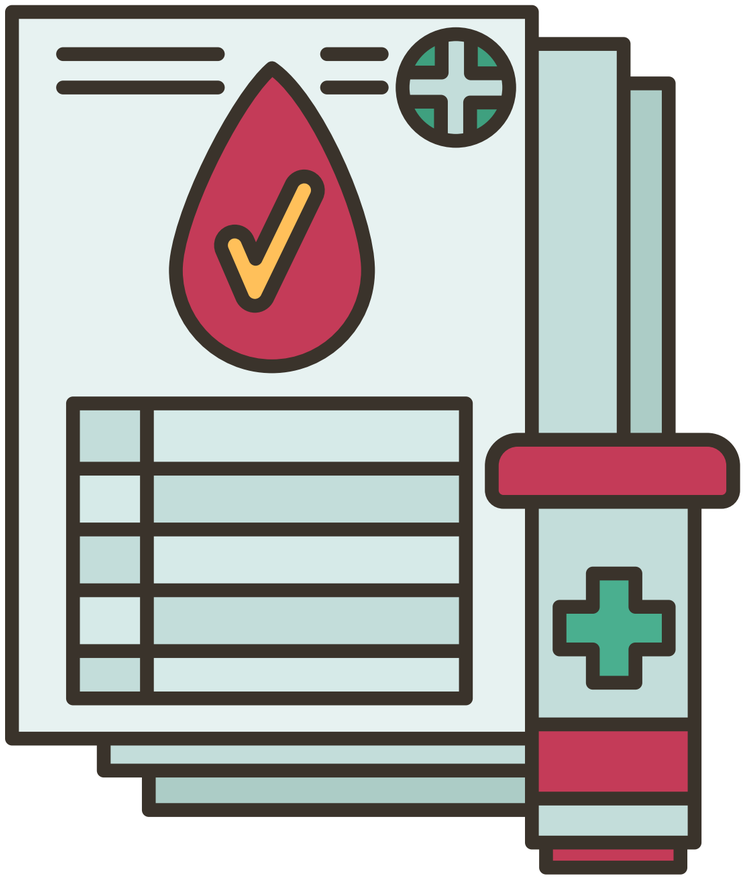
INTRODUCTION

A blood bank inventory and donor matcher system manages blood stock levels, tracks expiration dates, and connects eligible donors with patients in real-time. Reduces the time needed to find rare blood types during emergencies. Tracks expiration dates of perishable blood components like platelets and plasma. Replaces manual record-keeping with automated database tracking.





PROJECT OBJECTIVES



- Automate Donor Records: Register and maintain donor profiles (Name, Age, Blood Group, Contact Info) using custom classes and objects.
- Track Blood Inventory: Monitor available blood units across standard blood types (A+, A−, B+, B−, O+, O−, AB+, AB−) in real time.
- Match Donors & Patients: Process patient requests and instantly query stored records to check stock availability or compatible donor details using loop and conditional logic.
- Demonstrate Core Java Fluency: Efficiently apply foundational programming concepts like arrays, methods, conditional statements, and custom objects.

ADVANTAGES

- Instant Donor Matching: Quickly filters through donor records to pair matching blood groups during emergencies.
- Lightweight & Fast Execution: Runs directly in the Java Virtual Machine with zero external database overhead or heavy dependencies.



- Modular Codebase: Organizes responsibilities into separate classes and helper methods, making the codebase readable, reusable, and simple to debug
- Error Prevention: Replaces physical logbooks with systematic validation routines, reducing operational human errors in recording blood stock.

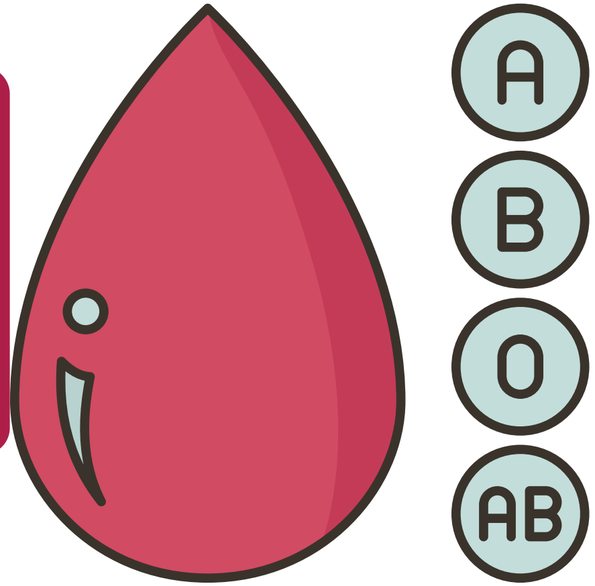
PROJECT DEVELOPMENT

- Blueprint Design: Created custom data models (Donor class, BloodBank management class).
- Logic Implementation: Developed core execution logic using conditional checks (if/else if), iterative processing (for loops), and structural 1D arrays.
- Menu Flow: Constructed a continuous interactive user loop driven by custom class methods and Scanner user inputs in the main program.

COURSE OUTCOMES (COS)

- CO1: Operational Fundamentals & Data Representation – Uses basic Java syntax, primitive data types (int, double, boolean), and object attributes (String).
- CO2: Control Structures & Iterative Logic – Employs conditional control flow (if, if-else) and loop constructs (for loops) to execute decision-making and search routines.

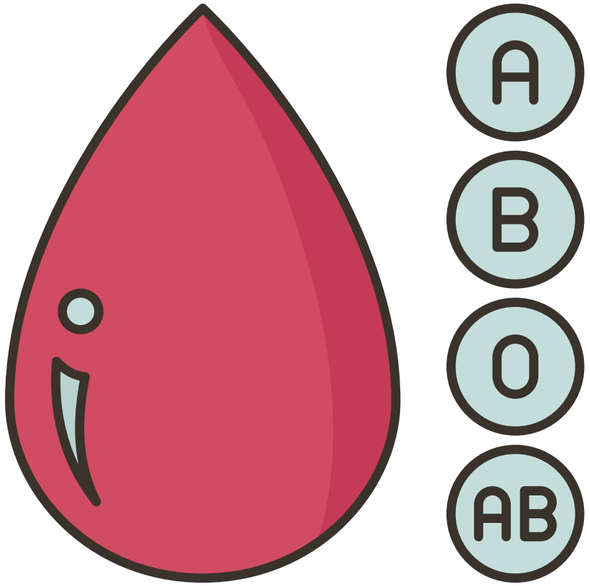
- CO3: Array Processing & Data Management – Utilizes single-dimensional (1D) arrays to sequentially store, iterate, search, and manage list data sets.



Course Outcomes (COs)

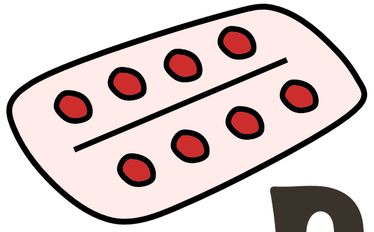
An ArrayList is a resizable collection in Java that stores multiple elements and allows adding, removing, and accessing elements dynamically.
A HashMap is a Java collection that stores data in key-value pairs, allowing quick retrieval of values using unique keys.

A String in Java is an object that represents a sequence of characters used to store and manipulate text. Strings are immutable, meaning their contents cannot be changed after creation.
Error correction in Java involves identifying, debugging, and fixing errors in a program. Java provides exception-handling mechanisms such as try, catch, finally, throw, and throws to manage runtime exceptions.



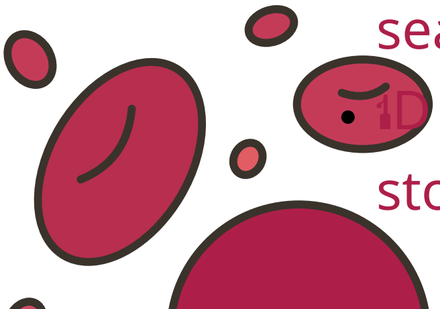
TOPICS COVERED

- Data Types
- Seletion Staments
- Itarration Staments
- Looping Statements
- Methods
- Arrays
- Hash maps
- String Methods



BACKGROUND

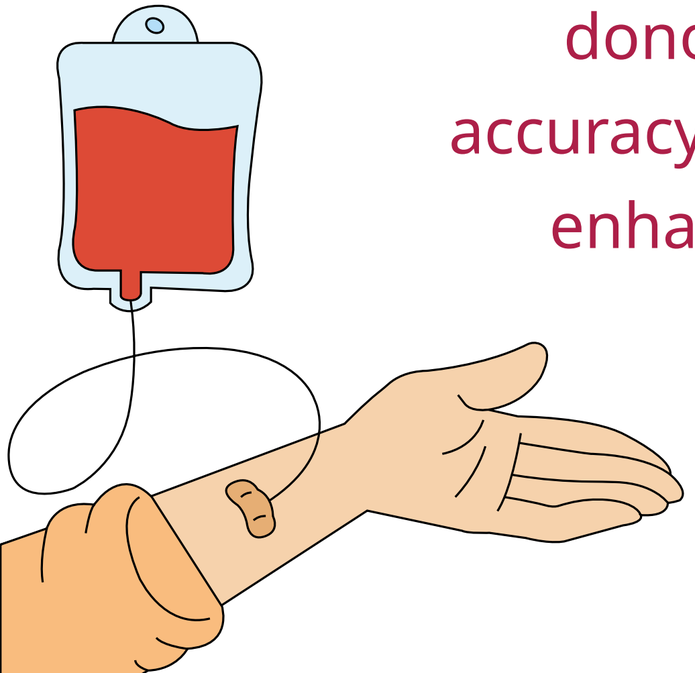
- Data Types: Used int, boolean, and String for donor and stock details.
- Classes & Objects: Used Donor class and objects to store donor information.
- Conditional Statements: Used if-else to check blood groups, menu choices, and stock.
- Loops: Used for loops to search and display donor/stock records.
- Methods: Used methods like addDonor(), searchBloodGroup(), and displayStock().
- 1D Arrays: Used arrays to store donor records and blood-stock data.

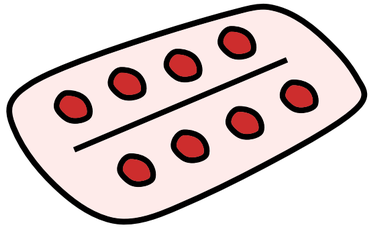




CONCLUSION

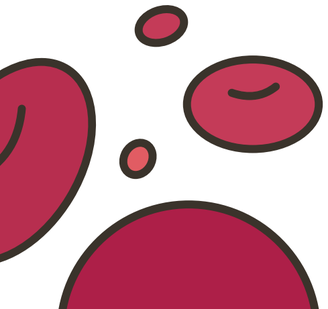
The Blood Bank Inventory and Donor Matching System provides an efficient way to manage blood stock and donor information.Using Java and OOP concepts, the system helps track blood availability and identify suitable donors quickly.It reduces manual work, improves accuracy, and supports faster blood management.Future enhancements can make the system more secure, automated, and user-friendly





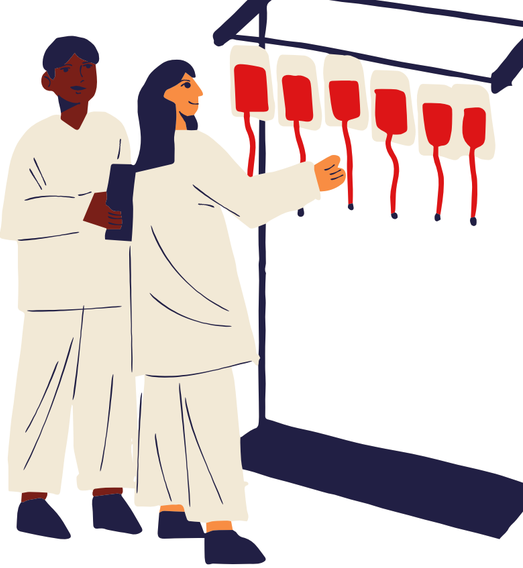
FUTURE SCOPE

- File I/O & Database Storage: Integrate Java File I/O (BufferedReader/FileWriter) or MySQL via JDBC to save donor and inventory logs permanently.
- Dynamic Collections: Replace standard 1D arrays with dynamic collection classes like ArrayList or HashMap for flexible array sizing and fast lookup speeds.
- GUI / Web Interface: Build a graphical front-end using Java Swing, JavaFX, or HTML/CSS to enable intuitive point-and-click operations.
- SMS Alert System: Integrate third-party API tools to send automated SMS notifications to registered donors when matching blood requests arrive.









THANK YOU

